Ders 7 Genişletilmiş Notu: Düzenli İfadeler, Düzenli İfade ile NFA Arası Dönüşümler ve GNFA

Ders 7 Genişletilmiş Notu: Düzenli İfadeler, Düzenli İfade ile NFA Arası Dönüşümler ve GNFA

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Düzenli İfadelere Dönüş
- 2. Düzenli İfade Bir Dil Tanımlar
- 3. Σ Gösterimi
- 4. İşlem Önceliği ve R^+ Gösterimi
- 5. Boş Küme ile İşlemler ve R^k Gösterimi
- 6. Düzenli İfadenin Tanımladığı Dili Bulma Örnekleri
- 7. Temel Özdeşliklerin Tekrarı
- 8. Düzenli İfadeler ile Sonlu Otomatların Denkliği
- 9. Düzenli İfadeden NFA'ya: $(ab \cup a)^*$
- 10. Düzenli İfadeden NFA'ya: $(a \cup b)^* aba$
- 11. NFA'dan Düzenli İfadeye: Sezgisel Yol
- 12. Genelleştirilmiş NFA (GNFA)
- 13. NFA'yı Özel GNFA'ya Çevirme
- 14. Durum Söküp Atma
- 15. Yöntemli Yolla Tam Örnek

1. Düzenli İfadelere Dönüş

Önceki haftalarda pompalama önsavını (pumping lemma) ve düzenli olmayan dilleri görmüştük. Düzenli ifadeleri (regular expressions) ise okumanız için size bırakmıştık. Hoca, dersin biraz hızlı ilerlediğini ve yeterince vakit olduğunu düşünerek bu konuya geri dönmeye karar verdi. Düzenli ifadeleri bitirdikten sonra bağlamdan bağımsız gramerlere (context-free grammars) devam edilecek.

Fikir aritmetikten geliyor. Aritmetikte $5 + 3 \times 4$ gibi ifadeler yazarız. Benzer şekilde, düzenli işlemleri (regular operations) kullanarak dil inşa edebiliriz. Bir aritmetik ifade bize bir sayı değeri verir. Bir düzenli ifade ise bize bir dil verir.

2. Düzenli İfade Bir Dil Tanımlar

2.1 İlk Örnek

Dersin ilk örneği şudur:

$$R = (0 \cup 1)0^*$$

Bu ifade hangi dili tanımlar? Dizgi 0 ya da 1 ile başlayabilir. Ardından bir miktar 0 gelebilir. Kleene yıldızı (Kleene star) hiç 0 gelmemesine de izin verir. Yani yıldızlı kısım boş dizgi (empty string, ε; hoca buna sık sık "null" diyor) olabilir. Böylece $L(R)$ dilinin ilk elemanları şunlardır:

$$0,\ 1,\ 00,\ 10,\ 000,\ 100,\ ...$$

Burada 0 dizgisi, "0 ve ardından ε" olarak düşünülebilir. 1 dizgisi de aynı şekilde. Sonra 1'in ve 0'ın arkasına istenen sayıda 0 eklenir.

2.2 Semboller Aslında Kümedir

Birleşim (union) bir küme işlemidir. O hâlde $0 \cup 1$ yazınca 0 ve 1 birer kümedir. Bu yazım kısaltmadır: 0, tek elemanı 0 sembolü olan $\{0\}$ kümesini, 1 de $\{1\}$ kümesini gösterir. İki sembol arasında hiçbir işaret yoksa o işlem birleştirmedir (concatenation). Dolayısıyla ifadenin açık yazımı adım adım şöyledir:

$$(0 \cup 1)0^* \ = \ (\{0\} \cup \{1\})\,\{0\}^* \ = \ \{0,1\}\,\{0\}^*$$

Önce $\{0\}$ ile $\{1\}$'in birleşimi alınır ve $\{0,1\}$ kümesi çıkar. Sonra bu küme, $\{0\}$ kümesinin Kleene yıldızıyla birleştirilir. Baştaki kısa yazım, işte bu uzun yazımın kısaltmasıdır.

2.3 Birleşim İçin $+$ İşareti

Bazı yerlerde birleşim yerine artı işareti görürsünüz: $0 \cup 1$ ile $0 + 1$ aynı şeydir. Bu biraz kafa karıştırabilir. Çünkü normalde $0{+}1$ size 1 verir. Burada ise $+$, birleşimin başka bir gösterimidir; toplama değildir.

Bu yazımın bir kolaylığı vardır: Parantezleri toplama ve çarpmadaki gibi açabilirsiniz.

$$(0 + 1)0^* \ = \ 00^* + 10^*$$

2.4 Kümelerde Birleştirme

Küme birleştirmesinde yeni bir küme kurulur. Birinci kümedeki her eleman için, o eleman yazılır ve arkasına ikinci kümeden bir eleman eklenir. Örneğin:

$$\{a, b\}\{c\} = \{ac, bc\}$$

Bunu $+$ yazımıyla hatırlamak çok kolaydır. a veya b olduğu için $(a + b)$ yazarız, arkasına da c gelir. Sonra çarpma yapar gibi dağıtırız:

$$(a + b)c = ac + bc$$

> **Ders sonrası not:** Hoca bu çarpımı söylerken "ac artı ab" dedi; doğrusu $ac + bc$'dir. Biraz önceki küme yazımında da sonuç $\{ac, bc\}$ idi.

2.5 0^* Ne Demektir?

0^*, $\{0\}$ kümesinin Kleene yıldızıdır. Bu kümede boş dizgi de vardır, sonsuza kadar giden 0 dizileri de:

$$0^* = \{\varepsilon, 0, 00, 000, ...\}$$

Yani sıfır tane dahil, istenen sayıda 0.

3. Σ Gösterimi

Σ alfabeyi (alphabet) gösterir. $\Sigma = \{0, 1\}$ ise düzenli ifadelerin içinde Σ'yı da kullanabiliriz. Hocanın örneği:

$$0\Sigma^*$$

Başta tek bir 0 vardır. Arkasından alfabeden istenen sayıda eleman gelir. Σ^* tek başına alfabeden üretilebilen bütün dizgileri verir:

$$\Sigma^* = \{\varepsilon, 0, 1, 00, 01, 10, 11, ...\}$$

Bunların hepsinin önüne 0 eklersek şunu buluruz:

$$0\Sigma^* = \{0, 00, 01, 000, 001, 010, 011, ...\}$$

Demek ki $0\Sigma^*$, 0 ile başlayan bütün dizgilerdir.

Σ burada $(0 + 1)$ demektir, yani $0\Sigma^* = 0(0 + 1)^*$ yazılabilir. Fakat bu parantez az önceki gibi açılamaz, çünkü üzerinde Kleene yıldızı vardır. O yüzden ifade böyle bırakılır: 0 ile başlayıp 0 ve 1'lerle devam eden dizgiler.

4. İşlem Önceliği ve R^+ Gösterimi

Düzenli işlemlerin bir öncelik sırası (order of precedence) vardır:

1. Önce Kleene yıldızı,
2. Sonra birleştirme,
3. En son birleşim.

Üst simge olarak yıldız yerine bazen $+$ da görülür. Bu üstteki $+$, birleşimden farklıdır: En az bir tekrar demektir. Örneğin 1^+ en az bir tane 1 içerir. Önce bir 1 yazılır, ardından 1^* gelir:

$$1^+ = 11^*$$

Genel olarak $R^+ = RR^*$'dır.

5. Boş Küme ile İşlemler ve R^k Gösterimi

5.1 Boş Kümeyle Birleşim

Düzenli ifadelerde boş küme (empty set, $\emptyset$) de kullanılabilir. Boş küme, hiç elemanı olmayan kümedir. $a + \emptyset$ yazıldığında, a'ların kümesi ile boş kümenin birleşiminden söz ediyoruz.

> **Ders sonrası not:** Hoca burada $a \cup \emptyset$ için "boş küme verir" dedi; doğrusu $a \cup \emptyset = a$'dır. Boş kümeyle birleşim etkisizdir. Boş küme veren işlem, aşağıdaki birleştirmedir. Doğru özdeşlikler Bölüm 7'de toplanmıştır.

5.2 Boş Kümeyle Birleştirme

İlginç olan şudur: $a\emptyset$ nedir? Bu, $\{a\}$ kümesinin boş kümeyle birleştirilmesidir. Sonuçtaki elemanların birinci parçası birinci kümeden, ikinci parçası ikinci kümeden gelmek zorundadır. Birinci parça a olabilir. Fakat ikinci kümede hiç eleman yoktur. Bu yüzden sonuç kümesine hiçbir eleman yazamayız:

$$a\emptyset = \emptyset$$

5.3 R^k Gösterimi

Hocanın burada eklediği bir gösterim daha vardır: Üs olarak sayı da yazılabilir. 1^2 iki tane 1, 1^3 üç tane 1 demektir. Genel olarak 1^k, k tane 1 demektir. R^k de R'nin k kez birleştirilmesidir.

6. Düzenli İfadenin Tanımladığı Dili Bulma Örnekleri

Bu bölümdeki bütün örneklerde alfabe $\Sigma = \{0, 1\}$'dir. Görev, verilen düzenli ifadenin tanımladığı dili yazmaktır.

6.1 0^*10^*

Bu dilin dizgilerinde tek bir 1 vardır. Bu 1'den önce ve sonra istenen sayıda 0 gelebilir:

$$L(0^*10^*) = \{w \mid w \text{ tam olarak bir tane 1 içerir}\}$$

6.2 $(\Sigma\Sigma)^*$

$\Sigma\Sigma$, kümeden bir sembol ve ardından yine kümeden bir sembol demektir. Yani her seferinde iki sembol gelir. Üzerine Kleene yıldızı da gelince:

$$L((\Sigma\Sigma)^*) = \{w \mid w \text{ çift sayıda sembol içerir}\}$$

Sıfır da çift sayıdır. Hiç eleman yoksa bunu da çift sayarız, yani ε bu dildedir.

6.3 $(0 \cup \varepsilon)1^*$

Burada önemli bir tuzak vardır. ε (null), boş küme değildir. Programlamadaki `""` gibidir: Bir dizgidir, ama içinde hiç sembol yoktur. Birleştirmede de hiçbir sembol eklemez.

> **Hocanın vurgusu:** ε ile $\emptyset$'nin farklı olduğunu mutlaka anlayın. Bu ifadeden ε'u atıp 01^* yazmak caziptir, ama yanlıştır. ε boş küme değildir.

Doğru yol, 1^*'ı içeri dağıtmaktır:

$$(0 \cup \varepsilon)1^* \;=\; 01^* \cup \varepsilon 1^* \;=\; 01^* \cup 1^*$$

$\varepsilon 1^*$ birleştirmesinde ε atılabilir. Sonuç, "0 ile başlayıp 1'lerle devam eden dizgiler" ile "yalnız 1'lerden oluşan dizgiler"in birleşimidir. 01^* ifadesi ise ε, 1, 11 gibi dizgileri içermez. İki ifadenin farkı budur.

6.4 $\emptyset^*$

Hoca kitaptaki örneklere bakarken ilginç bir sonucu gösterdi: Boş kümenin Kleene yıldızı boş dizgiyi verir.

$$\emptyset^* = \{\varepsilon\}$$

Kitabın açıklaması şöyledir: Yıldız işlemi, dilden istenen sayıda dizgiyi yan yana getirerek sonuçtaki bir dizgiyi elde eder. Dil boşsa, yıldız işlemi yine de sıfır tane dizgiyi yan yana getirebilir. Bu da yalnız boş dizgiyi verir.

Aradan döndükten sonra hoca bu sonucun kafa karıştırıcı olduğunu, kendisinin de henüz tam oturtamadığını, bir tanımdan geliyor gibi göründüğünü söyledi ve üzerinde durmadı.

> **Ders sonrası not:** Sonuç gerçekten tanımdan gelir. Kitaptan okunan cümlede de söylendiği gibi yıldız işlemi sıfır dizgi seçmeye izin verir: $L^* = \{x_1 x_2 \cdots x_k \mid k \geq 0,\ x_i \in L\}$. $k = 0$ için sonuç boş dizgidir. Bu seçim hiçbir x_i gerektirmediği için $L = \emptyset$ olsa bile $\varepsilon \in L^*$'dır. $k \geq 1$ için ise boş dilden eleman seçilemez, bu yüzden başka dizgi çıkmaz.

7. Temel Özdeşliklerin Tekrarı

Hoca aradan sonra bu temel özellikleri topladı:

İşlem	Sonuç	Açıklama
$R \cup \emptyset$	R	Boş kümeyle birleşim etkisizdir.
$R\varepsilon$	R	Boş dizgiyle birleştirme etkisizdir.
$R \cup \varepsilon$	R'ye eşit olabilir de olmayabilir de	Dikkat edilmesi gereken durum.
$R\emptyset$	$\emptyset$	İkinci parça için eleman yoktur.

> **Ders sonrası not:** Altyazıda hoca birinci satırı "düzenli ifadenin boş dizgiyle (empty string) birleşimi kendisini verir" diye söylüyor; doğrusu boş kümeyle ($\emptyset$) birleşimdir, yani $R \cup \emptyset = R$. Hemen ardından kendisi de $R \cup \varepsilon$'un R'ye eşit olmayabileceğini vurguluyor.

Aynı kurala göre, örneğin $1^*\emptyset = \emptyset$'dir: Birinci parça 1^*'dan gelebilir, ama ikinci parça için eleman yoktur.

> **Ek bilgi (derste anlatılmadı):** "Eşit olabilir de olmayabilir de" ifadesinin ölçütü şudur: $R \cup \varepsilon = R$ eşitliği, ancak ve ancak $\varepsilon \in L(R)$ ise doğrudur. ε zaten dildeyse birleşim dile yeni bir şey katmaz. Dilde değilse $R \cup \varepsilon$ fazladan ε içerir.

8. Düzenli İfadeler ile Sonlu Otomatların Denkliği

Düzenli ifadelerle ilgili en önemli nokta, sonlu otomatlara (finite automata) denk olmalarıdır:

$$\text{Bir dil düzenlidir} \iff \text{onu tanımlayan bir düzenli ifade vardır.}$$

Bu önermede "ancak ve ancak" (if and only if) olduğu için iki yön de geçerlidir. Her sonlu otomat bir düzenli ifadeye, her düzenli ifade de bir sonlu otomata dönüştürülebilir.

Böylece düzenli diller için elimizde üçüncü bir tanım daha olmuş olur. Düzenli bir dilin onu kabul eden bir DFA'sı, onu kabul eden bir NFA'sı ve onu tanımlayan bir düzenli ifadesi vardır.

> **Hocanın vurgusu:** Bir dilin düzenli olduğunu kanıtlamak için üç yol vardır: Onu kabul eden bir DFA tasarlarsınız, bir NFA tasarlarsınız ya da onu tanımlayan bir düzenli ifade yazarsınız. Düzenli ifade göstermek de yeterlidir.

Bundan sonra iki dönüşüm yönü de örneklerle görülecek.

9. Düzenli İfadeden NFA'ya: $(ab \cup a)^*$

Elimizde $(ab \cup a)^*$ ifadesi olsun. Bunu NFA'ya (non-deterministic finite automaton) çevirmek istiyoruz. Yöntemli yol, en küçük alt ifadelerden başlayıp makineyi adım adım büyütmektir. Hocanın çizdiği makineler aşağıda metin olarak verilmiştir; durum adları bu nottaki gösterim içindir.

9.1 Yapı Taşları

Tek sembol. a tek başına bir düzenli ifadedir. Onun NFA'sı iki durumludur: Başlangıç durumundan a okununca kabul durumuna gidilir. b için de aynısı geçerlidir.

Birleştirme (ab). a makinesinin kabul durumu, kabul durumu olmaktan çıkarılır. Bu durumdan b makinesinin başlangıcına null geçişi (ε-geçişi, null transition) konur. Dört durumlu zincir şöyledir:

$$q_2 \xrightarrow{a} q_3 \xrightarrow{\varepsilon} q_4 \xrightarrow{b} q_5$$

Birleşim ($ab \cup a$). a için makinemiz zaten var, fakat bu kez birleştirme değil birleşim gerekiyor. Ya ab kabul edilecek ya da a. Bu, paralel bir durum gibidir. Paralelliği yaratmak için yeni bir başlangıç durumu ekleriz ve ondan iki null geçişi çıkarırız. Üstteki kola ab makinesi, alttaki kola a makinesi yazılır.

Yıldız ($(ab \cup a)^*$). Kleene yıldızı, bu ifadeye uyan parçaların sıfır, bir ya da daha çok kez art arda kabul edilmesi demektir. Tekrar sonsuz olabileceği için bir döngü gerekir. Her kabul durumundan başlangıç durumuna bir null geçişi çizeriz.

Fakat burada bir şey eksik kaldı. İfade boş dizgiyi kabul etmeli, oysa bu ana kadarki makine onu kabul etmiyor. Bunun için en başa, kabul durumu olan yeni bir başlangıç durumu ekleyip onu null geçişiyle eski başlangıca bağlarız. Böylece boş dizgi de kabul edilir.

9.2 Ortaya Çıkan Makine

Sonuç sekiz durumlu şu NFA'dır. Başlangıç durumu q_0, kabul durumları q_0, q_5 ve q_7'dir.

Geçiş	Görevi
$q_0 \xrightarrow{\varepsilon} q_1$	Yeni başlangıç (boş dizgi için)
$q_1 \xrightarrow{\varepsilon} q_2$, $q_1 \xrightarrow{\varepsilon} q_6$	Birleşimin iki kolu
$q_2 \xrightarrow{a} q_3$, $q_3 \xrightarrow{\varepsilon} q_4$, $q_4 \xrightarrow{b} q_5$	ab kolu
$q_6 \xrightarrow{a} q_7$	a kolu
$q_5 \xrightarrow{\varepsilon} q_1$, $q_7 \xrightarrow{\varepsilon} q_1$	Yıldızın geri dönüşleri

Bu makine Python simülasyonuyla 12 sembole kadar bütün dizgilerde $(ab \cup a)^*$ ile karşılaştırıldı, fark çıkmadı. Yeni başlangıç durumu eklenmeden önce tek farkın ε olduğu da aynı denemede görüldü.

Hoca bu sonucun çok fazla durum verdiğini söyledi. Çünkü bu mekanik bir yöntemdir.

9.3 Ara Null Geçişi Gerekli mi?

Soru: a ile b arasında neden null geçişi var? a'dan doğrudan b'ye, oradan da kabul durumuna geçsek hatalı mı olur?

Cevap: Hatalı olmaz. Bu yöntem aslında bir bilgisayar programının dönüşümü nasıl yapacağını açıklar. Bir insan bunu yazsa, senin kurduğun gibi mantıklarla çok daha basit makineler yazabilir. Bu metot genel olarak gereğinden fazla durum verir, makineyi gereğinden karmaşık hâle getirir.

Bu öneri simülasyonla da denendi. q_3 ile q_4 tek duruma indirilince yedi durumlu makine aynı dili kabul eder.

Mekanik yöntemin neden epsilonlu kurulduğu konusunda ise hoca emin olmadığını, üzerinde pek düşünmediğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Mekanik yöntemin null geçişlerini koruması büyük olasılıkla genellik içindir. Yöntem her alt makineye, içeriğine bakmadan aynı biçimde uygulanmalıdır. Bir makinenin kabul durumundan çıkan oklar varsa, o durumu sonraki makinenin başlangıcıyla tek duruma indirmek dile istenmeyen dizgiler katabilir. Null geçişi ise iki makinenin içini birbirine karıştırmadan bağlar.

9.4 Sadeleştirme

Çoğu zaman elde edilen sonlu otomat sadeleştirilebilir. Kitap, bu ifade için yalnız iki durumlu bir sürüm bulunabileceğini söylüyor. Hoca bunun evde çalışılabileceğini belirttikten sonra derste kısaca denedi. Bu kez mekanik gitmiyoruz. Yıldız olduğu için boş dizgi kabul edilmelidir, o yüzden başlangıç durumu kabul durumu olur. ab de a da kabul edilmelidir. NFA olduğu için birden çok yol yazabiliriz. Sonra bu yollardan başa dönülür. Hoca önce üç durumlu bir makineye geldi, sonra döngüler koyarak durumları azaltmaya çalıştı. Sonucu da evde üzerinde çalışmaya bıraktı.

> **Ders sonrası not:** Derste iki durumlu makine sözlü olarak tamamlanmadı. Kısa notta verilen iki durumlu şu NFA doğrudur: q_0 hem başlangıç hem kabul durumudur; geçişler $q_0 \xrightarrow{a} q_0$, $q_0 \xrightarrow{a} q_1$, $q_1 \xrightarrow{b} q_0$'dır. Simülasyonla 14 sembole kadar bütün dizgilerde $(ab \cup a)^*$ ile aynı sonucu verdi.

10. Düzenli İfadeden NFA'ya: $(a \cup b)^*aba$

İkinci örnekte de aynı adımları izliyoruz.

Adım 1: $(a \cup b)^*$**.** a ve b için birer makine kurulur. $a \cup b$ için, yeni bir durumdan iki null geçişiyle bu makinelere gidilir. Yıldız için başa yeni bir kabul eden başlangıç durumu eklenir ve iki kabul durumundan birleşimin başına null geçişleri çizilir. Bu makinede altı durum vardır:

$$p_0 \xrightarrow{\varepsilon} p_1, \quad p_1 \xrightarrow{\varepsilon} p_2, \quad p_1 \xrightarrow{\varepsilon} p_4, \quad p_2 \xrightarrow{a} p_3, \quad p_4 \xrightarrow{b} p_5, \quad p_3 \xrightarrow{\varepsilon} p_1, \quad p_5 \xrightarrow{\varepsilon} p_1$$

Kabul durumları p_0, p_3 ve p_5'tir.

Adım 2: aba**.** Bu kısım basittir. Mekanik olarak üç küçük makine null geçişleriyle bağlanır:

$$r_0 \xrightarrow{a} r_1 \xrightarrow{\varepsilon} r_2 \xrightarrow{b} r_3 \xrightarrow{\varepsilon} r_4 \xrightarrow{a} r_5$$

Hoca mekanik yolu böyle yazdı, ama bu null geçişlerinin atlanabileceğini de söyledi. O zaman makine doğrudan a, b, a zinciridir.

Adım 3: Birleştirme. $(a \cup b)^*$ makinesinde üç kabul durumu var. Bunu aba makinesiyle nasıl birleştireceğiz? Bu kabul durumlarını olduğu gibi bırakamayız. Her birinden aba makinesinin başlangıcına bir null geçişi çizeriz: $p_0, p_3, p_5 \xrightarrow{\varepsilon} r_0$. Sonra üçü de kabul durumu olmaktan çıkarılır. Tek kabul durumu r_5 kalır.

> **Hocanın vurgusu:** Genel olarak iki makine birleştirilirken birinci makinenin kabul durumları kabul olmaktan çıkarılır ve her biri null geçişiyle ikinci makinenin başlangıcına bağlanır.

Bu on iki durumlu makine, ara null geçişleri atılmış on durumlu sürümüyle birlikte, simülasyonda 12 sembole kadar bütün dizgilerde $(a \cup b)^*aba$ ile aynı sonucu verdi. Eski kabul durumları bırakılsaydı makine ε, a, b gibi dile ait olmayan dizgileri de kabul ederdi.

Hoca bu makinenin de bariz biçimde sadeleştirilebileceğini söyledi, ama üzerinde durmadı.

11. NFA'dan Düzenli İfadeye: Sezgisel Yol

Şimdi ters yöne geçiyoruz: NFA'yı düzenli ifadeye çevirmek. İki yol vardır:

- **Yöntemli yol (methodical):** Bütün NFA'lar için kullanılabilir.
- **Sezgisel yol (intuitive):** Basit NFA'lar için kullanılabilir.

Sezgisel yol için aslında öğretilecek bir şey yoktur. Makineye bakıp ifadeyi yazarsınız.

11.1 Birinci Örnek

Başlangıç durumunda a ile bir döngü, oradan da b ile bir kabul durumuna geçiş olsun. Başlarken istenen sayıda a okunabilir, çünkü döngü vardır. Döngü Kleene yıldızıyla gösterilir. Kabul durumuna gitmek için de bir b gerekir. Sonuç:

$$a^*b$$

11.2 İkinci Örnek

Kısa nottaki okumaya göre iki durum da kabul durumudur: $q_0 \xrightarrow{a} q_1$, $q_1 \xrightarrow{b} q_0$; başlangıç durumu q_0'dır. Bu örnek göründüğünden biraz daha karmaşıktır, ama sezgisel olarak yine denenebilir.

Hoca şöyle ilerledi. Hiçbir şey yoksa boş dizgi vardır ve kabul edilir. a kabul edilir. ab de kabul edilir, istenen sayıda ab de. Ayrıca ilk a'dan sonra başka bir döngü daha vardır: a ile başlayıp ikinci duruma gelince b ve a ile dönülebilir.

Hoca burada kaybolduğunu söyleyip ara verdi. Aradan sonra şu fikirle döndü: ab ile başlanabilir ve bunlardan sıfır ya da daha fazla olabilir. Ya da önce ilk a'ya gidilir ve döngüler oradan başlar. Yani her durum bir başlangıç noktası gibi davranabilir.

> **Hocanın vurgusu:** Sezgisel yol, döngüler varsa çok hızlı büyür. NFA karmaşıksa yöntemli yolu kullanmanız gerekebilir.
>
> **Ders sonrası not:** Hoca aradan sonra $(ab{+}ba)^*{+}a(ba{+}ab)^*$ yazdı (ifade kısa nottaki kare okumasından alındı); bu ifade yanlıştır. Makinede q_0'dan b geçişi olmadığı için b ile başlayan hiçbir dizgi kabul edilmez. Oysa ifade ba, $baba$ ve $abba$ gibi dizgileri de üretir. Doğrusu $(ab)^*(a \cup \varepsilon)$'dur. Hocanın "her durum başlangıç noktası gibi" fikri de doğru yazılınca $(ab)^* \cup a(ba)^*$ verir, bu da aynı dildir. Simülasyonda 10 sembole kadar hocanın ifadesi 83 dizgide makineden farklı çıktı (ör. ba, aab, $bababababa$). Doğru iki ifade ise 14 sembole kadar hiçbir dizgide farklı çıkmadı.

12. Genelleştirilmiş NFA (GNFA)

12.1 Tanım

Yöntemli yolda önce NFA, GNFA (generalized non-deterministic finite automaton, genelleştirilmiş NFA) denen bir makineye çevrilir. Sonra geçişler parçalanıp yeniden birleştirilir.

GNFA'da geçişlerin üstünde düzenli ifadeler bulunabilir. Tek bir sembol yerine, oklara düzenli ifade yazılabilir. Fikir şudur: Makine tek sembol okumak yerine girdiden bu örüntünün sonuna kadar okur. Geri kalan her şey her zamanki gibi nondeterministiktir; örneğin ε-geçişleri de olabilir.

12.2 Özel Biçim

NFA'yı düzenli ifadeye çevirmek için GNFA'nın özel bir biçimine ihtiyacımız var. Özel biçimin koşulları şunlardır:

- **Başlangıç durumu:** Diğer bütün durumlara giden oku vardır, ama kendisine ok yoktur.
- **Kabul durumu:** Yalnız bir tane vardır ve başlangıç durumundan farklıdır. Diğer bütün durumlardan ona ok gelir, ondan hiçbir yere ok çıkmaz. Bir tür son durumdur.
- **Diğer durumlar:** Başlangıç ve kabul dışındaki her durumdan diğer her duruma ve kendisine birer ok vardır.

Yani elimizde bir başlangıç, bir kabul durumu ve aradaki durumlar vardır. Ara durumlar birbirine tam bağlıdır (fully connected). Başlangıçtan her birine ok gider, her birinden kabul durumuna ok gelir.

> **Ders sonrası not:** Hoca başlangıç durumu için yalnız "kendisine ok yok" dedi; koşul daha geniştir. Kısa notta da belirtildiği gibi başlangıç durumuna **hiçbir** durumdan

ok gelmez. Bir sonraki bölümdeki yeni başlangıç durumu ekleme adımı bunu zaten sağlar.

Bu çok kalabalık bir çizge olur. Hocaya göre bir makine için bu sorun değildir. Elle çalışırken de sadeleştirebiliriz: Örneğin etiketi boş küme olan okları çizmeyebiliriz.

13. NFA'yı Özel GNFA'ya Çevirme

İlk işimiz NFA'yı özel GNFA'ya çevirmektir. Bu kolaydır:

1. Yeni bir başlangıç durumu ekle ve ε-geçişiyle eski başlangıca bağla.
2. Yeni tek bir kabul durumu ekle. Eski kabul durumlarını kabul olmaktan çıkar ve hepsini ε-geçişiyle yeni kabul durumuna bağla.
3. Aynı iki durum arasında birden çok etiket varsa bunları birleşimle tek oka indir. Örneğin a, b etiketi $a \cup b$ olur.
4. İki durum arasında ok yoksa, $\emptyset$ etiketli bir ok ekle.

Üçüncü adımı hoca aşağıdaki örneğin içinde gösterdi. Dördüncü adımdaki boş geçişler hiçbir zaman kullanılamaz. Bu yüzden özel GNFA'ya çevirirken dil aynı kalır.

14. Durum Söküp Atma

Özel GNFA elde edildikten sonra başlangıç ve kabul dışındaki durumlar tek tek kaldırılır (rip off), bunu yaparken geçişler birleştirilir. Yalnız başlangıç ve kabul durumu kalana kadar devam edilir. Dikkat: Kaldırılan şey ok değil, durumdur.

Üç durum düşünelim: q_i, q_j ve kaldıracağımız q_{rip}. Etiketler şöyle olsun:

- $q_i \xrightarrow{R_1} q_{rip}$
- q_{rip} üzerinde döngü: R_2
- $q_{rip} \xrightarrow{R_3} q_j$
- $q_i \xrightarrow{R_4} q_j$

q_i'den q_j'ye iki türlü gidilir. Birinci yol doğrudan R_4'tür. İkinci yol q_{rip} üzerindendir: Önce R_1 ile q_{rip}'e gidilir, orada R_2 ile istenildiği kadar dönülür, sonra R_3 ile çıkılır. Yani $R_1 R_2^* R_3$. q_{rip} kaldırılınca bu iki seçenek tek oka birleştirilir:

$$q_i \xrightarrow{R_4 \cup R_1 R_2^* R_3} q_j$$

Bu işlem bütün (q_i, q_j) çiftleri için yapılır. Özetle yöntem üç adımdır: NFA'dan özel GNFA elde et; durumları kaldır ve geçişleri birleştir; yalnız başlangıç ve kabul durumu kaldığında aradaki okun etiketini oku. Aranan düzenli ifade odur.

> **Ek bilgi (derste anlatılmadı):** Boş küme özdeşlikleri bu kuralda işe yarar. q_{rip}'te döngü yoksa $R_2 = \emptyset$'dir ve $\emptyset^* = \{\varepsilon\}$ olduğundan $R_1 R_2^* R_3 = R_1 R_3$ kalır. R_1 ya da R_3 boş kümeyse $R\emptyset = \emptyset$ kuralıyla o yol tümden kaybolur. $R_4 = \emptyset$ ise $R \cup \emptyset = R$ kuralıyla yalnız diğer yol kalır. Bu yüzden elle çözerken $\emptyset$ okları çizilmese de sonuç değişmez.

15. Yöntemli Yolla Tam Örnek

Dersin son örneği küçük bir makinedir. Kısa nottaki okumaya göre şöyledir: 1 başlangıç durumu, $1 \xrightarrow{a} 1$, $1 \xrightarrow{b} 2$, $2 \xrightarrow{a,b} 2$, 2 kabul durumu.

Adım 1: Özel GNFA. 2 üzerindeki iki etiket veya anlamındadır. Düzenli ifadeye çevirdiğimiz için $a \cup b$ yazarız. Yeni başlangıç s ve yeni kabul durumu acc eklenir:

$$s \xrightarrow{\varepsilon} 1, \qquad 1 \xrightarrow{a} 1, \qquad 1 \xrightarrow{b} 2, \qquad 2 \xrightarrow{a \cup b} 2, \qquad 2 \xrightarrow{\varepsilon} acc$$

Hoca zaman alacağı için bazı adımları atladı.

Adım 2: 2 durumunu kaldırma. $R_1 = b$, $R_2 = a \cup b$, $R_3 = \varepsilon$, $R_4 = \emptyset$ alınır (ara adım eklendi):

$$1 \xrightarrow{b(a \cup b)^*} acc$$

1'deki a döngüsü yerinde kalır.

Adım 3: 1 durumunu kaldırma. $R_1 = \varepsilon$, $R_2 = a$, $R_3 = b(a \cup b)^*$, $R_4 = \emptyset$ alınır (ara adım eklendi):

$$s \xrightarrow{a^* b(a \cup b)^*} acc$$

Geriye yalnız başlangıç ve kabul durumu kaldı. Düzenli ifade:

$$a^* b (a \cup b)^*$$

Yani bir miktar a'dan sonra bir b, ardından a ve b'lerden oluşan herhangi bir devam. Durum sökme işlemi Python'da her iki sırayla (2 sonra 1; 1 sonra 2) çalıştırıldı. İki sonuç da 12 sembole kadar bütün dizgilerde makineyle aynı sonucu verdi.

> **Ders sonrası not:** Altyazıda sonuç "$a^* b (a \cup b) b^*$" gibi geçiyor; doğrusu $a^* b (a \cup b)^*$'dır. Altyazıdaki biçim b dizgisini kabul etmez, oysa makine b'yi kabul eder.
>
> **Ders sonrası not:** Bölüm 11.2'deki makineye de aynı yöntem uygulanabilir (derste yapılmadı). q_1 kaldırılınca q_0 üzerinde ab döngüsü oluşur ve $q_0 \to acc$ etiketi $\varepsilon \cup a$ olur. q_0 kaldırılınca $(ab)^*(\varepsilon \cup a)$ çıkar. Bu, sezgisel yolda kaybolan ifadenin doğru hâlidir.
>
> **Hocanın vurgusu:** Bu konuyu normalde kendi başınıza okumanız istenmişti, yani bunları zaten biliyor olmanız bekleniyor. Vakit olduğu için derste üzerinde biraz daha çalışıldı.

Dersin Toparlaması

Düzenli ifadeler, düzenli işlemlerle (birleşim, birleştirme, Kleene yıldızı) dil tanımlamanın bir yoludur. Öncelik sırası yıldız, birleştirme, birleşimdir. ε ile $\emptyset$ birbirinden farklıdır ve bu fark $R \cup \emptyset = R$, $R\varepsilon = R$, $R\emptyset = \emptyset$ ve $\emptyset^* = \{\varepsilon\}$ gibi özdeşliklerde ortaya çıkar. Bir dil ancak ve ancak onu tanımlayan bir düzenli ifade varsa düzenlidir.

Bu denkliğin iki yönü de yapıcı olarak görüldü. Düzenli ifadeden NFA'ya giderken küçük makineler null geçişleriyle büyütülür; yöntem mekaniktir ve gereğinden çok durum üretir. NFA'dan düzenli ifadeye giderken basit makinelerde sezgi yeterlidir, fakat sezgisel yol hızla büyür. Genel yol, NFA'yı özel GNFA'ya çevirip $R_4 \cup R_1 R_2^* R_3$ kuralıyla durumları teker teker kaldırmaktır. Bundan sonra bağlamdan bağımsız gramerlere devam edilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.